

Contexto

Você fará parte de um time responsável por construir um sistema de controle de carteira de um FIDC. Neste teste, você receberá um arquivo `carteira.csv` com dados fictícios de direitos creditórios e deverá processá-los, validá-los e gerar relatórios estruturados.

Arquivo de Entrada

O arquivo `carteira.csv` será disponibilizado junto com este enunciado. Ele contém dados fictícios de direitos creditórios com os seguintes campos:

Campo	Tipo esperado
<code>id</code>	inteiro
<code>cedente</code>	string
<code>cpf_cnpj</code>	string
<code>sacado</code>	string
<code>valor_nominal</code>	numérico
<code>data_aquisicao</code>	data
<code>data_vencimento</code>	data
<code>status</code>	string — <code>a_vencer</code> , <code>vencido</code> , <code>liquidado</code> , <code>inadimplente</code>
<code>numero_parcela</code>	inteiro

Entregáveis

- `processamento.py` — pipeline principal (leitura, limpeza, análise)
 - `modelos.py` — classes e modelagem OOP
 - `relatorio.json` — relatório gerado pelo script
 - `RESPOSTAS.md` — respostas escritas obrigatórias
-

Bloco 1 — Leitura e Limpeza dos Dados

- Ler o arquivo `carteira.csv` usando a biblioteca de sua preferência
 - Identificar e reportar no console **todos os problemas encontrados nos dados**
 - Tratar cada problema encontrado e documentar as decisões tomadas
 - Ao final, os dados devem estar limpos, padronizados e prontos para processamento
-

Bloco 2 — Validação de CPF e CNPJ

- Validar o campo `cpf_cnpj` de todos os cedentes da carteira

- O uso de bibliotecas externas para validação (como `validate-docbr`, `pycpfcpnpj` ou similares) é **vetado**. Implemente os algoritmos de validação de CPF e CNPJ manualmente
 - Registros com documentos inválidos devem ser mantidos na carteira, mas sinalizados com uma flag `documento_invalido = True`
-

Bloco 3 — Análise da Carteira

Com os dados limpos e validados, calcular:

- Valor total da carteira
 - Valor e quantidade de títulos **vencidos** (baseado na `data_vencimento` comparada com a data de 01/01/2026)
 - **Taxa de inadimplência**: percentual do valor nominal dos títulos com `status == "inadimplente"` sobre o total
 - Agrupamento de valor total e quantidade por `status`
 - Agrupamento de valor total por `cedente`
 - Identificar títulos com **inconsistência lógica** entre campos — o enunciado não especifica quais são; cabe ao candidato definir e implementar as validações que julgar pertinentes
-

Bloco 4 — Modelagem Orientada a Objetos

Em `modelos.py`, implementar as seguintes classes:

`DireitoCreditorio`

- Atributos: `id`, `cedente`, `cpf_cnpj`, `sacado`, `valor_nominal`, `data_aquisicao`, `data_vencimento`, `status`, `numero_parcela`
- Método `esta_vencido()` → `bool`
- Método `dias_em_atraso()` → `int` (0 se não vencido)
- Método `tem_inconsistencia()` → `bool` (detecta conflito lógico entre campos)
- Método `inconsistencias()` → `list[str]` (descrição de cada inconsistência encontrada neste registro)

`Carteira`

- Atributos: `nome`, `direitos` (lista de `DireitoCreditorio`)
 - Método `valor_total()` → `float`
 - Método `taxa_inadimplencia()` → `float` (percentual)
 - Método `titulos_vencidos()` → lista de `DireitoCreditorio`
 - Método `relatorio_por_cedente()` → dicionário com valor total e quantidade por cedente
 - Método `inconsistencias()` → lista de direitos com ao menos um conflito lógico
-

Bloco 5 — Geração do Relatório JSON

Gerar `relatorio.json` com:

- `data_geracao`: data e hora de execução

- `nome_carteira`: nome da carteira
 - `total_registros`: após limpeza
 - `valor_total`: soma dos valores nominais
 - `taxa_inadimplencia_pct`: percentual formatado com 2 casas decimais
 - `resumo_por_status`: quantidade e valor por status
 - `resumo_por_cedente`: valor total e percentual sobre a carteira por cedente
 - `alertas`: lista descritiva com todos os problemas encontrados — documentos inválidos, zeros restaurados, inconsistências lógicas, registros removidos ou corrigidos na limpeza
-

RESPOSTAS.md — Perguntas Obrigatórias

Responda com suas próprias palavras. As respostas serão confrontadas diretamente com o código entregue na etapa de revisão. Respostas genéricas ou desconexas do código enviado serão desconsideradas.

1. No Bloco 1, quais problemas você encontrou nos dados? Liste cada um e descreva exatamente como tratou e por quê tomou essa decisão.
 2. Explique o algoritmo que você implementou para validar CPF e CNPJ. Como funciona o cálculo dos dígitos verificadores em cada caso?
 3. Quais inconsistências lógicas entre campos você implementou no Bloco 3 e no método `tem_inconsistencia()`? Por que escolheu essas e não outras?
 4. No método `taxa_inadimplencia()` da classe `Carteira`, o que acontece se a carteira estiver vazia? Como você tratou esse caso?
 5. Se você precisasse adaptar este pipeline para rodar diariamente de forma automatizada, o que mudaria na sua implementação?
 6. Se a carteira tivesse 10 milhões de linhas, o que mudaria na sua abordagem de leitura, validação e geração do relatório?
-

Observações Finais

- O uso de IA não é vetado formalmente, mas as respostas do `RESPOSTAS.md` e uma eventual revisão ao vivo serão usadas para avaliar o real entendimento do candidato sobre cada decisão tomada.
- **Toda decisão não óbvia no código deve ser justificada** — seja num comentário inline, no `RESPOSTAS.md` ou em ambos.
- Ambiguidades no enunciado são intencionais. A decisão tomada e sua justificativa são tão importantes quanto o resultado final.